



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации информационных
технологий

Отчет по практическим работам №9-10

по дисциплине «Системная и программная инженерия»

Выполнили:

Студенты группы ИКБО-22-23

Петров А.
Михеев Н.В.
Моисеев А.М.
Плюснин И.И.

Проверил:

Мельников Д.А.

2026 г.

Содержание

ВВЕДЕНИЕ	3
ПРАКТИЧЕСКАЯ РАБОТА № 9. ОРГАНИЗАЦИЯ РАЗРАБОТКИ	5
9.1 Задание к практической работе	5
9.2 Выбор методологии управления процессом разработки	5
9.3 Обоснование выбранного подхода	6
9.4 Удаленный git-репозиторий и стратегия ветвления	7
9.5 Инструменты разработки программного обеспечения	8
9.6 Организация работы команды и процесс поставки изменений	9
ПРАКТИЧЕСКАЯ РАБОТА № 10. ДОКУМЕНТИРОВАНИЕ РАЗРАБОТКИ ПО	11
10.1 Задание к практической работе	11
10.3 Структура и состав документации разработчика	12
10.4 Документация пользователя	14
10.6 Примеры пользовательских инструкций	15
ЗАКЛЮЧЕНИЕ	17
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	18

ВВЕДЕНИЕ

Настоящий отчёт составлен по результатам выполнения практических занятий № 9–10 по дисциплине «Системная и программная инженерия». Работа продолжает ранее подготовленные и успешно сданные отчёты по практическим работам № 1–4 и № 5–8, в которых были определены предметная область, пользовательские и функциональные требования, нефункциональные ограничения, архитектура, логическая модель данных и техническое задание на веб-приложение «Генератор технических заданий для учебных проектов».

Цель текущего этапа заключалась в формализации организационной стороны разработки и подготовке эксплуатационной документации. На практике № 9 был выбран и обоснован подход к управлению процессом разработки, описаны средства совместной работы команды, удалённый репозиторий и инструменты, применяемые на этапах программирования, тестирования и сопровождения. На практике № 10 рассмотрены два вида документации: документация разработчика, необходимая для сопровождения и развития программного продукта, и документация пользователя, обеспечивающая понятное освоение основных сценариев работы с системой.

Содержание отчёта согласовано с ранее зафиксированным планом реализации в Trello, где проект развивался по спринтам, а роли участников были распределены между менеджером проекта, backend-разработчиком, тестировщиком-аналитиком и frontend-разработчиком-дизайнером. Поэтому в качестве организационной модели разработки выбран итерационный подход с элементами Scrum, наиболее естественно продолжающий уже сложившуюся практику работы команды.

Дополнительно учтено ограничение на использование Fandom при выполнении практической работы № 10. В качестве платформы пользовательской документации выбрана GitHub Wiki, поскольку она естественно интегрируется с git-репозиторием проекта, поддерживает

версионирование, позволяет хранить инструкции в Markdown и обеспечивает единое пространство для сопровождения продукта.

ПРАКТИЧЕСКАЯ РАБОТА № 9. ОРГАНИЗАЦИЯ РАЗРАБОТКИ

9.1 Задание к практической работе

Задание практической работы № 9 включает следующие пункты:

- 1) выбрать методологию управления процессом разработки выбранного проекта исходя из потребностей команды;
- 2) создать удалённый git-репозиторий на одном из популярных сервисов;
- 3) описать выбранные инструменты разработки программного обеспечения.

9.2 Выбор методологии управления процессом разработки

Для проекта «Генератор технических заданий для учебных проектов» выбрана итерационно-инкрементная модель разработки с элементами Scrum. Такой выбор обусловлен характером самого продукта: функциональность приложения удобно развивать по частям, получая работоспособный результат уже после каждой итерации. В проекте можно отдельно доводить до готовности модули авторизации, генерации ТЗ, экспорта документов, истории запросов и формирования шаблона Trello, не дожидаясь абсолютного завершения всей системы.

С учётом ранее принятого плана работ команда фактически уже была организована по спринтам. В отчёте по практическим работам № 1–4 задачи были распределены на три спринта: аналитика и проектирование, реализация ключевых модулей, тестирование и подготовка документации. Следовательно, переход к итерационному подходу не требует перестройки организационной логики, а, наоборот, формализует тот режим работы, который был выбран на старте проекта.

В рамках выбранной модели каждая итерация завершается конкретным промежуточным результатом. По итогам Sprint 1 были получены требования, диаграммы и первичная модель системы; Sprint 2 ориентирован на разработку backend, frontend, генератора T3, экспорта DOCX/PDF и интеграции с Trello; Sprint 3 охватывает тестирование, исправление дефектов, подготовку технической и пользовательской документации. Такое разбиение обеспечивает прослеживаемость между требованиями, архитектурой и фактическими задачами команды.

9.3 Обоснование выбранного подхода

Каскадная модель для данного проекта признана менее подходящей. Несмотря на понятную линейную структуру, она неудобна для небольшого учебного веб-приложения, требования к которому уточняются по ходу работы. Веб-интерфейс, шаблоны генерации, форматы экспорта и состав документации целесообразно уточнять постепенно, с быстрым возвратом к предыдущим решениям. При жёстком waterfall-подходе такие изменения приводили бы к лишним согласованиям и удорожанию процесса.

Спиральная модель также не была выбрана, поскольку она лучше подходит для проектов с выраженным управлением рисками, высокой стоимостью ошибок и большим числом витков уточнения архитектуры. Для учебного проекта из четырёх человек подобный процесс оказался бы избыточным. Полноценные тяжеловесные методологии наподобие RUP тоже не соответствуют масштабу системы: они создают значительные накладные расходы на формальные артефакты и роли.

Итерационно-инкрементный подход с элементами Scrum оказался наиболее сбалансированным вариантом. Он сочетает короткие циклы поставки результата, регулярную синхронизацию команды, возможность приоритизации задач в бэклоге и удобное распределение ответственности. Для проекта, где уже

используются Trello, спринтовое планирование и разделение ролей, такой подход обеспечивает управляемость разработки без лишней бюрократии.

В качестве практических элементов Scrum в проекте используются бэклог, спринты длительностью 2–4 недели, назначение ответственных за задачи, контроль статусов «Бэклог», «Sprint», «В работе», «Готово», а также короткие командные синхронизации по ходу итерации. При этом модель остаётся адаптированной под учебный формат и не воспроизводит Scrum в полном объёме.

9.4 Удаленный git-репозиторий и стратегия ветвления

В качестве платформы для удалённого репозитория выбран GitHub. Решение обусловлено тем, что сервис широко распространён, поддерживает совместную работу с ветками и pull request, позволяет хранить issues, wiki-страницы и историю изменений, а также хорошо интегрируется с Visual Studio Code и другими инструментами разработки.

Репозиторий проекта имеет следующую логическую структуру: каталог frontend для клиентской части на React + TypeScript, каталог backend для серверной части API и бизнес-логики, каталог docs для технической и пользовательской документации, а также служебные файлы конфигурации, docker-compose, шаблоны окружения и инструкции по развёртыванию. Такое разбиение упрощает навигацию по проекту и облегчает сопровождение.

Для работы с кодовой базой используется упрощённая стратегия ветвления. Ветка main содержит стабильное состояние проекта. Ветка develop служит интеграционной веткой текущего спринта. Для отдельных задач создаются тематические ветки вида feature/auth, feature/spec-generator, feature/export-docx, feature/trello-template, fix/validation и docs/wiki. После завершения задачи изменения отправляются в pull request, проходят код-ревью и только затем

сливаются в develop. В main попадают версии, прошедшие интеграционную проверку и готовые к демонстрации.

Коммиты оформляются в едином стиле с указанием типа изменения: feat, fix, docs, refactor, test, chore. Это повышает читаемость истории и позволяет быстро отследить, какие изменения связаны с функциональностью, исправлениями, документацией или внутренним обслуживанием проекта.

Ветка	Назначение	Содержание изменений
main	Стабильная версия	Демонстрационные и принятые версии проекта
develop	Интеграция спринта	Объединение завершённых задач текущей итерации
feature/*	Функциональные задачи	Реализация отдельных модулей и пользовательских сценариев
fix/*	Исправления	Коррекция дефектов, найденных на тестировании
docs/*	Документация	Обновление отчётов, README и Wiki

Таблица 9.1 — Схема организации веток и назначение

9.5 Инструменты разработки программного обеспечения

Для реализации проекта используется набор инструментов, покрывающий управление задачами, написание кода, тестирование, проектирование интерфейса, контроль версий, документирование и развёртывание. Выбранные средства соответствуют архитектуре системы и ролям участников команды.

Категория	Инструмент	Ответственный	Назначение
Управление задачами	Trello	Илья	Ведение бэклога, планирование спринтов, контроль сроков и статусов задач
Контроль версий	Git + GitHub	Вся команда	Совместная разработка, хранение истории изменений, pull request и code review

Backend	FastAPI / Python	Никита	REST API, бизнес-логика генерации ТЗ, валидация и интеграция с БД
Frontend	React + TypeScript	Андрей	SPA-интерфейс, формы генерации, история ТЗ, экспорт и публикация
База данных	PostgreSQL	Никита	Хранение пользователей, шаблонов, сгенерированных ТЗ и публичных ссылок
Тестирование API	Postman	Артём, Никита	Проверка REST-методов, сценариев авторизации и ошибок валидации
Ручное тестирование UI	Google Chrome DevTools	Артём, Андрей	Проверка адаптивности, сетевых запросов и клиентских ошибок
Проектирование интерфейса	Figma	Андрей	Макеты экранов и согласование пользовательских сценариев
Контейнеризация	Docker / Docker Compose	Никита	Единое окружение для локального запуска и развёртывания
Документация	Markdown + GitHub Wiki + TypeDoc + Sphinx	Никита	Подготовка технической и пользовательской документации

Таблица 9.2 — Основные инструменты разработки

9.6 Организация работы команды и процесс поставки изменений

Организация процесса разработки опирается на ранее распределённые роли. Менеджер проекта ведёт бэклог и следит за сроками выполнения задач. Backend-разработчик реализует API, бизнес-логику генерации и интеграцию с PostgreSQL. Frontend-разработчик отвечает за интерфейс, маршрутизацию, формы и экспорт пользовательских данных. Тестировщик-аналитик проверяет сценарии работы, фиксирует дефекты, уточняет поведение системы и поддерживает актуальность диаграмм и описаний процессов.

Каждый спринт начинается с актуализации задач в Trello. Для задач, имеющих достаточную степень проработки, назначается исполнитель, срок и критерий готовности. В процессе работы исполнитель создаёт отдельную ветку, выполняет локальную разработку и прикладывает описание изменений к pull request. До слияния в общую ветку выполняется самопроверка, локальный запуск тестов и ручная валидация основного пользовательского сценария.

Поставка изменений в интеграционную ветку сопровождается кратким ревью со стороны второго участника команды. Для backend-модулей проверяются корректность маршрутов, валидация входных параметров, обработка ошибок и совместимость с базой данных. Для frontend-модулей контролируются адаптивность, связность экранов, корректность запросов к API и понятность интерфейса. После интеграции изменений тестировщик проводит регрессионную проверку ключевых сценариев: регистрация, авторизация, генерация ТЗ, экспорт в DOCX/PDF, просмотр истории и открытие публичной ссылки.

Такой процесс позволяет сохранять управляемость даже при небольшом составе команды. Работа остаётся прозрачной: для каждой задачи известны исполнитель, статус, связанная ветка и коммиты, а история обсуждений хранится непосредственно рядом с кодом и карточками задач.

ПРАКТИЧЕСКАЯ РАБОТА № 10. ДОКУМЕНТИРОВАНИЕ РАЗРАБОТКИ ПО

10.1 Задание к практической работе

Задание практической работы № 10 включает следующие пункты:

- 1) создать документацию разработчика при помощи любого инструмента для генерации документации кода;
- 2) создать документацию пользователя в формате wiki-проекта либо при помощи другой подходящей платформы.

10.2 Документация разработчика

Для документации разработчика выбраны два инструмента в соответствии с двухкомпонентной архитектурой системы. Клиентская часть (React + TypeScript) документируется с помощью TypeDoc — генератора документации для TypeScript, который по JSDoc-аннотациям в исходном коде автоматически строит описание модулей, интерфейсов, классов, методов и типов. Серверная часть (Python + FastAPI) документируется с помощью Sphinx с расширением autodoc — стандартного инструмента для Python-проектов, который извлекает описание из docstring-комментариев и формирует HTML-документацию по всем слоям backend: контроллерам (FastAPI-роутерам), сервисам бизнес-логики, репозиториям и Pydantic/SQLAlchemy-моделям.

Назначение документации разработчика — упростить сопровождение системы и ускорить подключение нового участника к работе. По документации должно быть ясно, какие модули существуют в системе, за что отвечает каждый сервис, какие параметры принимают методы, какие структуры данных передаются между frontend и backend, а также где находятся точки расширения продукта.

Документация разделена на две независимо генерируемые части. TypeDoc формирует сайт из каталога frontend/src и охватывает компоненты React,

TypeScript-интерфейсы, DTO-типы и перечисления. Sphinx с autodoc обходит каталог backend/ и документирует FastAPI-маршруты, сервисы, репозитории и модели данных. Общий каталог docs/ в репозитории хранит README с инструкцией по запуску обоих генераторов, схему переменных окружения и инструкции по типовым операциям: создание миграций, подключение тестовой базы, запуск backend и frontend в режиме разработки.

10.3 Структура и состав документации разработчика

Раздел	Что документируется	Практическое назначение	Инструмент
Обзор проекта	Назначение системы, стек, структура репозитория	Быстрое введение нового разработчика в проект	README.md
Frontend: компоненты	React-компоненты, TypeScript-интерфейсы, хуки	Понимание клиентской архитектуры и точек расширения	TypeDoc
Frontend: типы и DTO	Интерфейсы запросов и ответов, перечисления	Согласованность типов между frontend и backend	TypeDoc
Backend: сервисы	AuthService, GeneratorService, ExportService, TrelloService	Сопровождение и расширение бизнес-логики	Sphinx
Backend: API-маршруты	FastAPI-роутеры, параметры, коды ответов	Интеграция frontend и backend без расхождений	Sphinx
Backend: модели	SQLAlchemy-модели, Pydantic-схемы, связи таблиц	Понимание доменной модели и структуры БД	Sphinx
Окружение и сборка	Переменные окружения, Docker, миграции, запуск	Воспроизводимость локального окружения	README.md

Таблица 10.1 — Основные разделы документации разработчика и инструменты генерации

Для ключевых модулей подготовлены описания, отражающие их роль в архитектуре, зафиксированной в отчёте по практическим работам № 5–8. На стороне frontend документируются TypeScript-интерфейсы компонентов, типы входных и выходных данных (DTO), хуки и вспомогательные функции. На

стороне backend — методы сервисов: для AuthService — регистрация, проверка пароля, генерация и валидация JWT; для GeneratorService — выбор шаблона, подстановка параметров, формирование структуры ТЗ; для ExportService — подготовка файлов DOCX и PDF; для SharedLink — создание, валидация и разрешение публичных ссылок.

Примеры структуры комментариев для генерации документации приведены ниже — отдельно для frontend (TypeDoc / JSDoc) и backend (Sphinx / Google-style docstring).

```
// frontend/src/services/generatorService.ts
/**
 * Генерирует ТЗ по типу проекта и уровню сложности.
 * @param type - тип проекта: WEB | MOBILE | DESKTOP | GAMEDEV
 * @param complexity - уровень: LAB | COURSE_WORK | DIPLOMA
 * @param userId - id пользователя (необязательно для гостя)
 * @returns объект Specification с заполненными разделами
 */
async generateSpec(
  type: ProjectType,
  complexity: ComplexityLevel,
  userId?: string
): Promise<Specification>

# backend/services/generator_service.py
def generate_spec(
    spec_type: ProjectType,
    complexity: ComplexityLevel,
    user_id: str | None = None,
) -> Specification:
    """
    Генерирует техническое задание по шаблону.

    Args:
        spec_type: тип проекта (WEB, MOBILE, DESKTOP, GAMEDEV).
        complexity: уровень сложности (LAB, COURSE_WORK, DIPLOMA).
        user_id: идентификатор пользователя; None для гостевого режима.

    Returns:
        Объект Specification с заполненной структурой разделов.

    Raises:
        TemplateNotFoundError: если шаблон для комбинации не найден.
    """
    ...
```

TypeDoc обрабатывает JSDoc-комментарий и формирует страницу метода с описанием назначения, сигнатурой, типами параметров и возвращаемого значения. Sphinx с autodoc читает Google-style docstring и строит аналогичную

страницу для Python-функции: разделы Args, Returns и Raises отображаются в структурированном виде. Такой подход обеспечивает единообразное качество документации для обеих частей системы.

10.4 Документация пользователя

С учётом отдельного требования не использовать Fandom в данной работе в качестве платформы пользовательской документации выбрана GitHub Wiki. Такой формат позволяет хранить страницы рядом с проектом, фиксировать историю изменений, организовывать навигацию между разделами и оформлять инструкции в привычном Markdown-формате.

Пользовательская документация ориентирована на студентов, преподавателей и начинающих разработчиков, для которых важны простые пошаговые инструкции. Поэтому акцент сделан не на внутреннем устройстве системы, а на понятном описании сценариев: как зарегистрироваться, как выбрать тип проекта, как запустить генерацию ТЗ, как экспортировать результат, как получить публичную ссылку и что делать при ошибках ввода.

В отличие от документации разработчика, wiki-проект должен быть максимально прикладным и понятным без специальной подготовки. Для каждой функции выделяются цель, последовательность действий, ожидаемый результат, возможные ошибки и способы их устранения.

10.5 Структура GitHub Wiki проекта

Страница Wiki	Содержание	Назначение страницы
Home	Краткое описание сервиса и навигация	Стартовая страница и точка входа в справку
Регистрация и вход	Создание аккаунта, авторизация, восстановление доступа	Освоение сценария входа в систему
Создание ТЗ	Выбор типа проекта, уровня сложности, запуск генерации	Основной пользовательский сценарий

Экспорт результата	Скачивание DOCX/PDF и работа с историей	Получение итогового документа
Публичная ссылка	Создание и открытие share-ссылки	Передача ТЗ преподавателю или команде
Интеграция с Trello	Формирование шаблона задач	Переход от ТЗ к планированию разработки
FAQ	Ответы на типовые вопросы и ошибки	Снижение нагрузки на поддержку

Таблица 10.2 — Предлагаемая структура пользовательской wiki

10.6 Примеры пользовательских инструкций

Ниже приведён пример краткой структуры wiki-страницы «Создание ТЗ».

1. Открыть главную страницу сервиса и перейти к форме генерации.
2. Выбрать тип проекта: веб-приложение, мобильное приложение, корпоративная ИС или иная доступная категория.
3. Указать уровень сложности: лабораторная работа, курсовая работа или дипломный проект.
4. При необходимости уточнить дополнительные параметры: предметную область, предполагаемые функции, особенности интерфейса.
5. Нажать кнопку «Сгенерировать ТЗ» и дождаться формирования результата.
6. Проверить структуру документа на экране, после чего сохранить его в истории, скачать в DOCX/PDF либо опубликовать по ссылке.

Для страницы «FAQ» фиксируются наиболее вероятные проблемные ситуации: пустые обязательные поля, недопустимые значения параметров, истечение сессии пользователя, ошибка генерации файла экспорта, отсутствие доступа к ранее созданной публичной ссылке. Для каждой ситуации указывается причина, видимое сообщение системы и рекомендуемое действие пользователя.

Таким образом, пользовательская документация не дублирует техническое описание модуля, а сопровождает жизненный цикл работы с сервисом от первого

входа до получения готового технического задания и передачи результата другим участникам проекта.

ЗАКЛЮЧЕНИЕ

В ходе выполнения практической работы № 9 была формализована организационная модель разработки веб-приложения «Генератор технических заданий для учебных проектов». Для команды из четырёх человек выбран итерационно-инкрементный подход с элементами Scrum, соответствующий уже применяемому спринтовому планированию. Описаны удалённый git-репозиторий на GitHub, стратегия ветвления, набор инструментов разработки и порядок поставки изменений.

По итогам практической работы № 10 определены подходы к сопровождению программного продукта посредством двух видов документации. Для документации разработчика выбрана связка TypeDoc (frontend, TypeScript) и Sphinx (backend, Python), позволяющая формировать описание модулей, сервисов, типов и API по комментариям в исходном коде. Для документации пользователя выбрана GitHub Wiki, так как использование Fandom в данной работе исключено. Подготовлена структура страниц, охватывающая основные пользовательские сценарии системы.

Полученные результаты логически продолжают материалы практических работ № 1–8: требования, архитектура, компоненты и этапы разработки получили организационное и документальное завершение. Это делает проект не только спроектированным, но и подготовленным к коллективной реализации, сопровождению и демонстрации.

Ссылка на wiki проекта: https://github.com/asterix1221/generator_TZ/wiki/FAQ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 ГОСТ 7.32—2017. Система стандартов по информации, библиотечному и издательскому делу. Отчёт о научно-исследовательской работе. Структура и правила оформления. — М.: Стандартинформ, 2017. — 27 с.

2 ГОСТ 19.201-78. Техническое задание. Требования к содержанию и оформлению. — М.: Изд-во стандартов, 1978. — 6 с.

3 ГОСТ 34.602-2020. Техническое задание на создание автоматизированной системы. — М.: Стандартинформ, 2020. — 24 с.

4 Фаулер М. UML. Основы. — М.: Символ-Плюс, 2004. — 192 с.

5 Буч Г., Рамбо Дж., Джекобсон А. Язык UML. Руководство пользователя. — М.: ДМК Пресс, 2006. — 432 с.

6 Bass L., Clements P., Kazman R. Software Architecture in Practice. — 3rd ed. — Addison-Wesley, 2012. — 640 p.

7 Fowler M. Patterns of Enterprise Application Architecture. — Addison-Wesley, 2002. — 560 p.

8 PlantUML Official Documentation [Электронный ресурс]. — URL: <https://plantuml.com/ru/> (дата обращения: 01.03.2026).

9 FastAPI Official Documentation [Электронный ресурс]. — URL: <https://fastapi.tiangolo.com/> (дата обращения: 01.03.2026).

10 React Documentation [Электронный ресурс]. — URL: <https://react.dev/> (дата обращения: 01.03.2026).

11 PostgreSQL 16 Documentation [Электронный ресурс]. — URL: <https://www.postgresql.org/docs/16/> (дата обращения: 01.03.2026).

12 Trello Help Documentation [Электронный ресурс]. — URL: <https://support.atlassian.com/trello/> (дата обращения: 01.03.2026).